

计算理论

Theory of Computation

授课教师：郑 乾

qianzheng@zju.edu.cn

助教：李文耀

wenyaoli@zju.edu.cn

浙江大学玉泉校区教 4-302

课程助教

李文耀

- 邮箱: wenyaoli@zju.edu.cn
- 图灵2023级



课程引论

Course Introduction

■ Fundamental questions of

- Physics, Chemistry, Biology, Medicine, Mathematics...

■ Fundamental questions of **Computer Science**

- What is an algorithm?

- What is computable?

- How can we characterize the difficulty of computation?

■ The **purpose** of this course

- Introduce a formal and rigorous definition of these concepts

- **Turing Machine!!!**

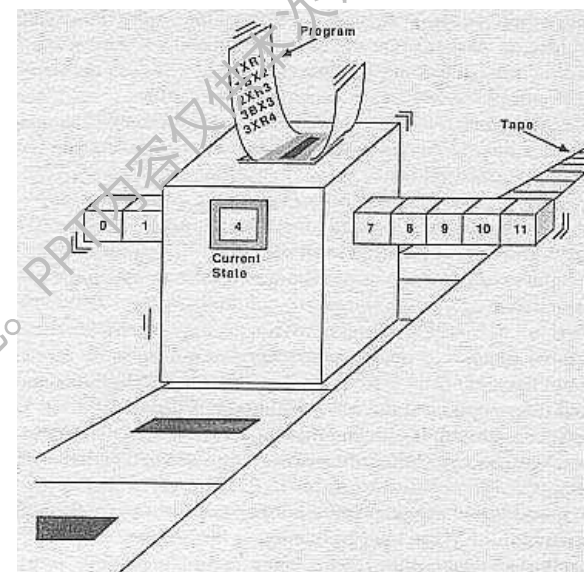
■ The **content** of this course: Turing Machine

- **Basic Concepts & Properties** of the Turing Machine

- **Foundations** for Turing Machine Invention

■ Turing Machine (definition from Wiki)

- A Turing machine is a mathematical model of computation describing an abstract machine that manipulates symbols on a strip of tape according to a table of rules. Despite the model's simplicity, it is capable of implementing any computer algorithm.

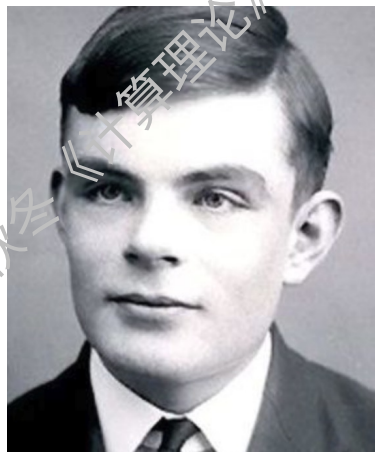
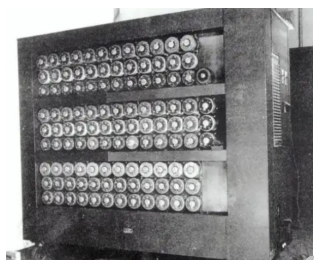


课程引论 (course introduction)



浙江大学 计算机科学与技术学院
COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY
ZHEJIANG UNIVERSITY

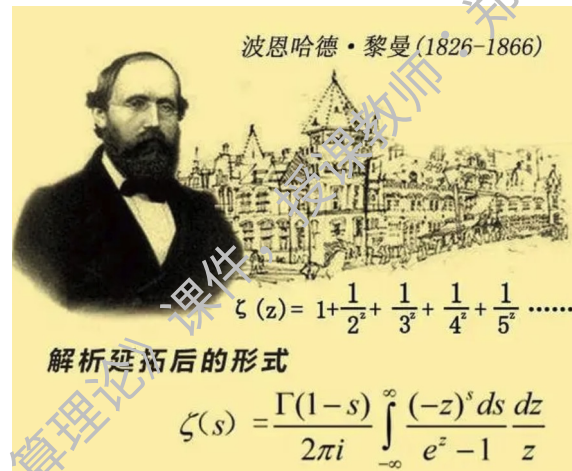
■ Turing Machine



ON COMPUTABLE NUMBERS, WITH AN
APPLICATION TO THE ENTSCHEIDUNGSPROBLEM

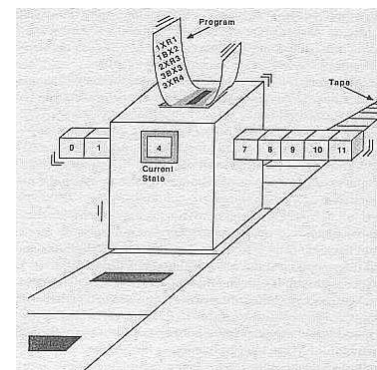
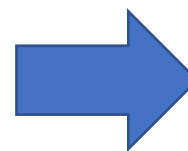
By A. M. TURING

[Received 28 May, 1936.—Read 12 November, 1936.]



Problem
Description

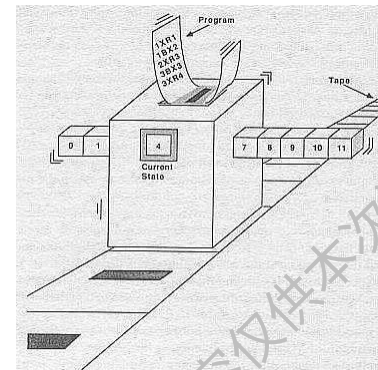
Problem
Symbolization



Find a
counterexample

■ OpenAI发布声明称：

- 一个比**GPT-6 Astra**更强、尚未公开的下一代模型，组织约**1万个AI智能体**，用**88小时**给出了纳维—斯托克斯方程存在性与光滑性问题的证明，并同步公开**166页论文**和**Lean**（计算机定理证明器）验证代码。
- 美国纽约大学教授特里斯坦·布克马斯特称，他与一名**Anthropic**研究员此前在相近方向得到带光滑外力的欧拉方程有限时间奇点，并完成**Lean**验证；因曾用**Codex**处理过对话，他质疑**OpenAI**模型是否接触会话影响了**训练数据**，并称对方曾提议由他单独发表。



GPT-6 Astra

It's out. Here's what it costs.



课程引论 (course introduction)



1. 计算的定义 (图灵机)



图灵机 (1936)

提出了计算的形式化模型，
奠定了“可计算性”的理论基础。

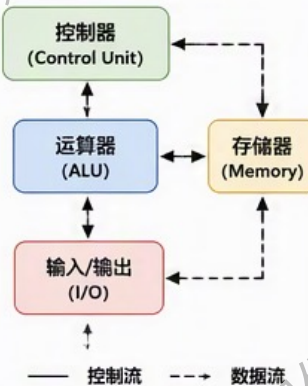
... 1 0 1 1 0 0 1 ...

有限状态控制器

核心思想：任何可计算的问题都可以被算法解决

2. 计算的实现 (基于冯氏架构的通用计算机)

冯·诺依曼架构



以存储程序和顺序执行为特征，
实现通用、可编程的计算。
支撑了过去数十年信息技术
产业的蓬勃发展。

特征：通用性强、编程灵活、易于规模化

3. 计算的应用 (以大模型为代表的AI等)

典型应用领域

- 大模型/生成式AI
- 计算机视觉
- 语音识别与处理
- 自动驾驶
- 生物医药
- ...

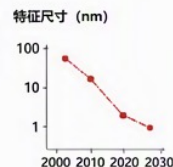
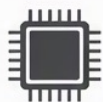


以大模型为代表的人工智能推动
计算需求指数级增长，改变生产力
并重塑社会生活。

价值：赋能千行百业，驱动社会智能化转型

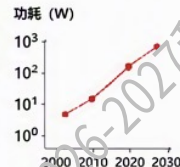
当前挑战：基于冯氏架构的计算实现面临物理极限

集成密度逼近极限



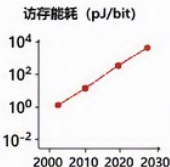
摩尔定律放缓，先进制程成本高昂，
继续缩小面临物理与经济双重瓶颈。

功耗墙日益严峻



泄漏功耗与动态功耗快速增长，
限制系统性能与能效比提升。

存储墙限制性能



存储与计算分离导致“冯氏瓶颈”：
数据搬运开销大、延迟高、能耗高。

❗ 冯氏架构的“存储-计算分离”是产生上述问题的根本原因

未来方向：类脑计算芯片有望突破瓶颈



- 高集成与并行**
大规模神经元/突触并行，突破传统制程缩放限制
- 低功耗高效能**
事件驱动、稀疏激活，更接近生物能效水平
- 存算一体**
在存储位置完成计算，显著降低数据搬运开销
- 更适合智能任务**
处理不确定性、时序信息，具备更强泛化与鲁棒性

类脑计算芯片有望从根本上突破冯氏架构的物理瓶颈，
为下一代智能计算带来新的突破与想象空间。

课程用书 (course book)



浙江大学

计算机科学与技术学院

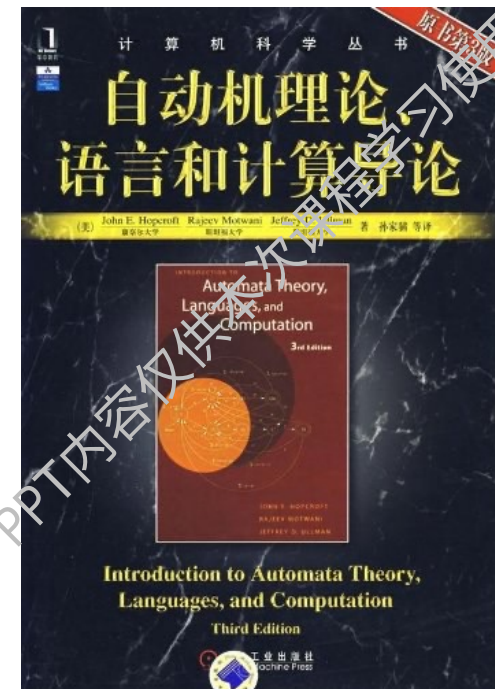
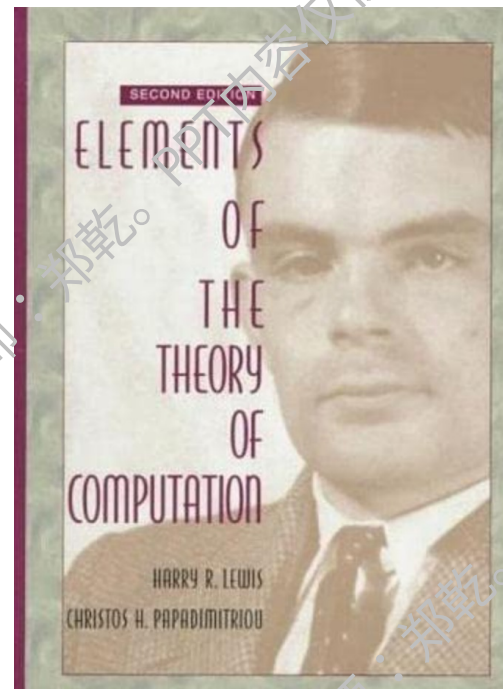
COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY
ZHEJIANG UNIVERSITY

教材

- Elements of The Theory of Computation (Second Edition), Harry R. Lewis, Christos H. Papadimitriou, Prentice Hall, 清华大学出版社

参考书

- 自动机理论、语言和计算导论（第3版），John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman, 清华大学出版社



- [Anita Wasilewska](https://www3.cs.stonybrook.edu/~cse303/) (Stony Brook University)
<https://www3.cs.stonybrook.edu/~cse303/>
- [Michael Sipser](https://ocw.mit.edu/courses/18-404j-theory-of-computation-fall-2020/pages/lecture-notes/) (MIT)
<https://ocw.mit.edu/courses/18-404j-theory-of-computation-fall-2020/pages/lecture-notes/>
- [Kemal Oflazer](https://www.andrew.cmu.edu/user/ko/pdfs/) (CMU)
<https://www.andrew.cmu.edu/user/ko/pdfs/>

课程内容 (course content)



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY
ZHEJIANG UNIVERSITY



algorithmic number theory

The **equivalent** relationship between computational model (**solution**) and language (**problem**)

Regular
Language

Context-free
Language

No context-free
Language

Finite
Automata

Pushdown
Automata

Turing
Machine

■ CH1: Basics (3 classes)

- Sets, Relations, and Language

■ CH2: Simple Models (3 classes)

- Regular Language and Finite Automata

■ CH3: Simple Models (3 classes)

- Context-free Languages and Pushdown Automata

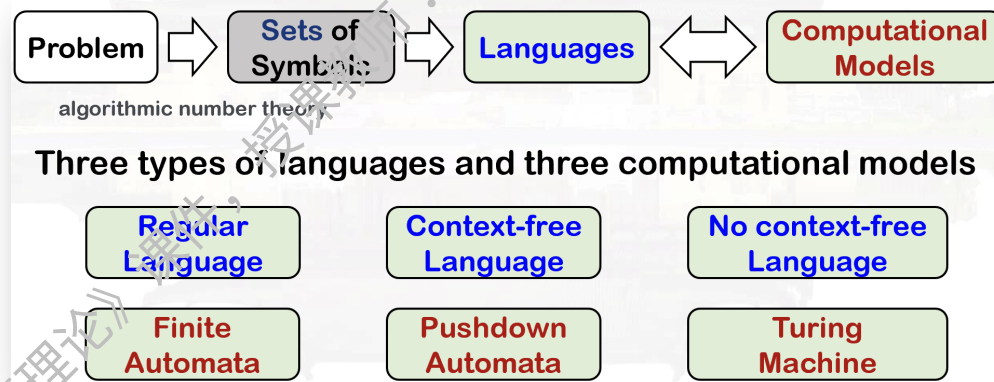
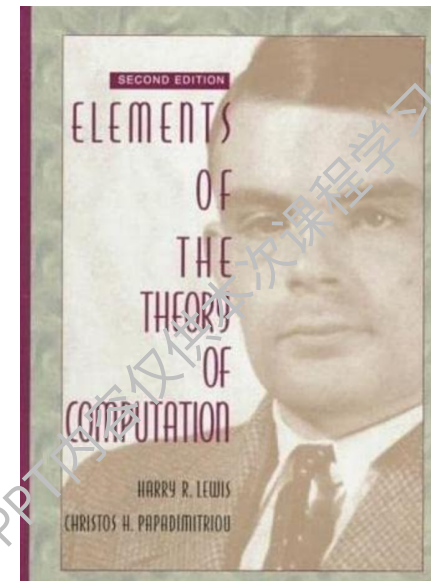
■ CH4: Turing Machine (3 classes)

- Turing Machine

■ CH5: Turing Machine (2 classes)

- Undecidability

■ Summary and Review (2 classes)





■ 平时成绩 30分

■ 期末笔试 70分

□ 短时间完成的直觉题(3次)

□ 2027年01月12日(08:00-10:00)

秋学期								冬学期								寒假		
九月			十月				十一月				十二月				一月			
一	二	三	四	五	六	七	八	一	二	三	四	五	六	七	八	考试周		
14	21	28	5	12	19	26	2	9	16	23	30	7	14	21	28	4	11	18
15	22	29	6	13	20	27	3	10	17	24	1	8	15	22	29	5	12	19
16	23	30	7	14	21	28	4	11	18	25	2	9	16	23	30	6	13	20
17	24	1	8	15	22	29	5	12	19	26	3	10	17	24	31	7	14	21
18	25	2	9	16	23	30	6	13	20	27	4	11	18	25	1	8	15	22
19	26	3	10	17	24	31	7	14	21	28	5	12	19	26	2	9	16	23
20	27	4	11	18	25	1	8	15	22	29	6	13	20	27	3	10	17	24

第1章 集合、关系、语言

Ch1. Sets, Relations, and Language

1.1 Sets



■ Set

□ An **unordered** collection of elements

□ Empty set $\emptyset$

■ Subsets and proper subsets

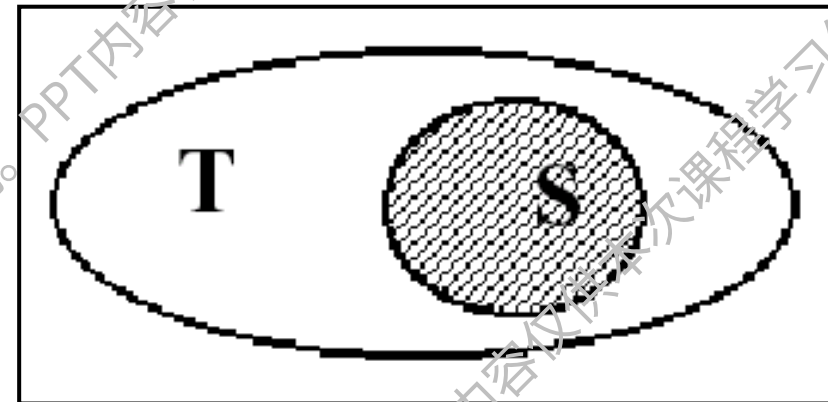
□ Subset notation: $\subseteq$

$$S \subseteq T \Leftrightarrow (\forall x \in S \Rightarrow x \in T)$$

□ Proper Subset: $\subset$

□ Two sets are Equal iff they contain the same elements

$$S = T \Leftrightarrow (S \subseteq T) \wedge (T \subseteq S)$$



➤ Is $\{1,2,3\}$ and $\{3,2,1\}$ the same set?

➤ Is $\{1,2,3\}$ a subset of $\{1,2,3\}$? Proper subset?

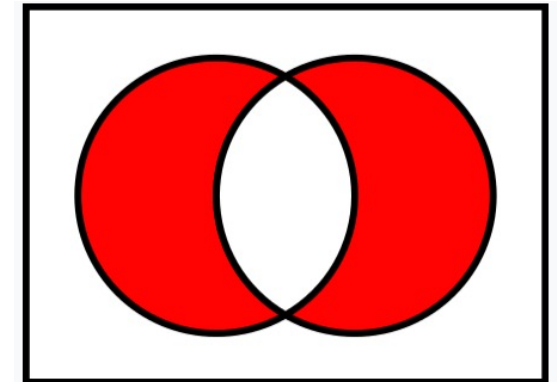
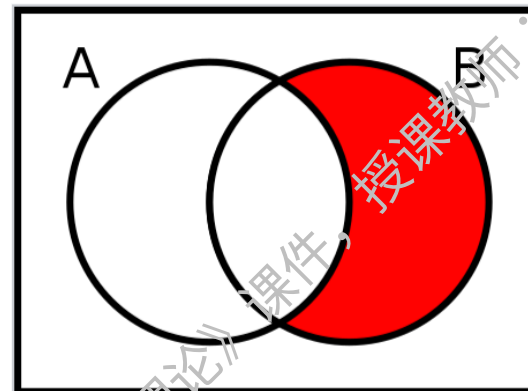
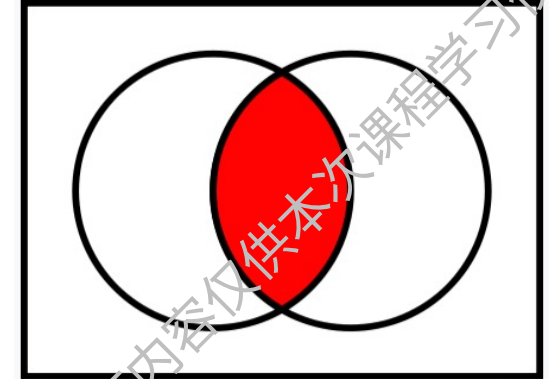
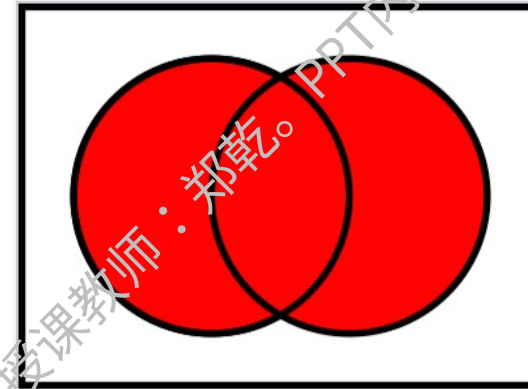
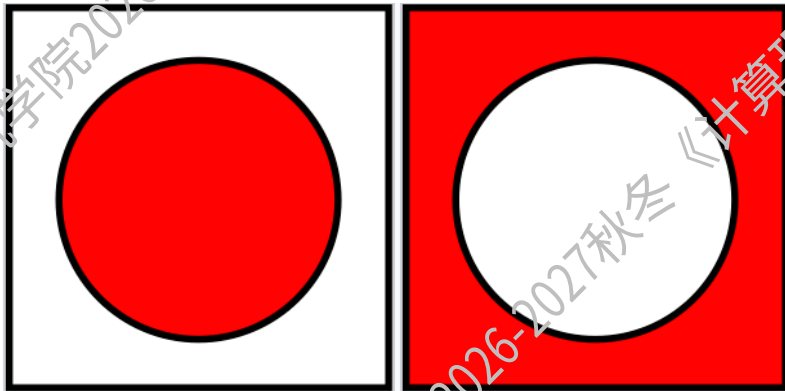
➤ How to define equal if we do not have the concept of subsets?

1.1 Sets



■ Set Operations

- Union: $A \cup B = \{x: x \in A \text{ or } x \in B\}$
- Intersection: $A \cap B$, disjoint
- Complement
- Difference: $B - A$
- Symmetric Difference



1.1 Sets



■ Set Identities

□ Idempotent Law

- $A \cap A = A, A \cup A = A$

□ Commutative Law

- $A \cup B = B \cup A, A \cap B = B \cap A$

□ Associative Law

- $A \cup (B \cap C) = (A \cup B) \cap C, A \cap (B \cup C) = (A \cap B) \cup C$

□ Distributive Law

- $A \cup (B \cap C) = (A \cup B) \cap (A \cup C), A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$

□ Absorption

- $A \cup (A \cap B) = A, A \cap (A \cup B) = A$

□ De Morgan's Law

- $A - (B \cup C) = (A - B) \cap (A - C), A - (B \cap C) = (A - B) \cup (A - C)$

■ Why some problems can be solved by employing computational models?

□ Problem $\Leftrightarrow$ Sets or language

□ Automated solution $\Leftrightarrow$ problem has Identities $\Leftrightarrow$ can be computed



algorithmic number theory

Three types of languages and three computational models

Regular Language

Context-free Language

No context-free Language

Finite Automata

Pushdown Automata

Turing Machine

1.1 Sets



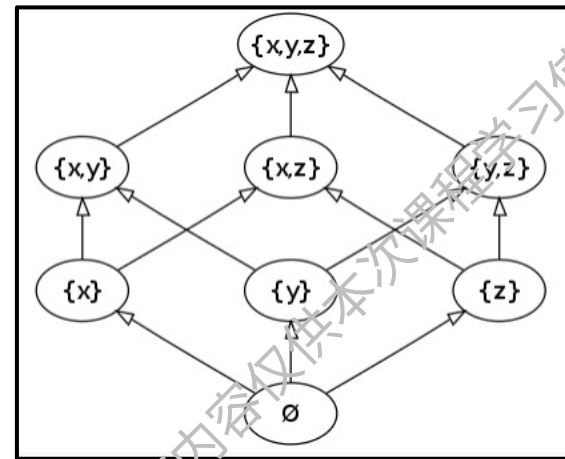
■ Power Set

□ 2^A = Set of all subsets of A

- $2^A = \{T \mid T \subseteq A\}$

□ Example

- If S is the set $\{x, y, z\}$, then all the subsets of S are
 - $\{\}$ (also denoted $\emptyset$, the empty set)
 - $\{x\}, \{y\}, \{z\}$
 - $\{x, y\}, \{x, z\}, \{y, z\}$
 - $\{x, y, z\}$
- Hence the power set of S is $\{\emptyset, \{x\}, \{y\}, \{z\}, \{x, y\}, \{x, z\}, \{y, z\}, \{x, y, z\}\}$



➤ Think about the rationale behind employing this notation for the Power Set.

1.1 Sets



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY
ZHEJIANG UNIVERSITY

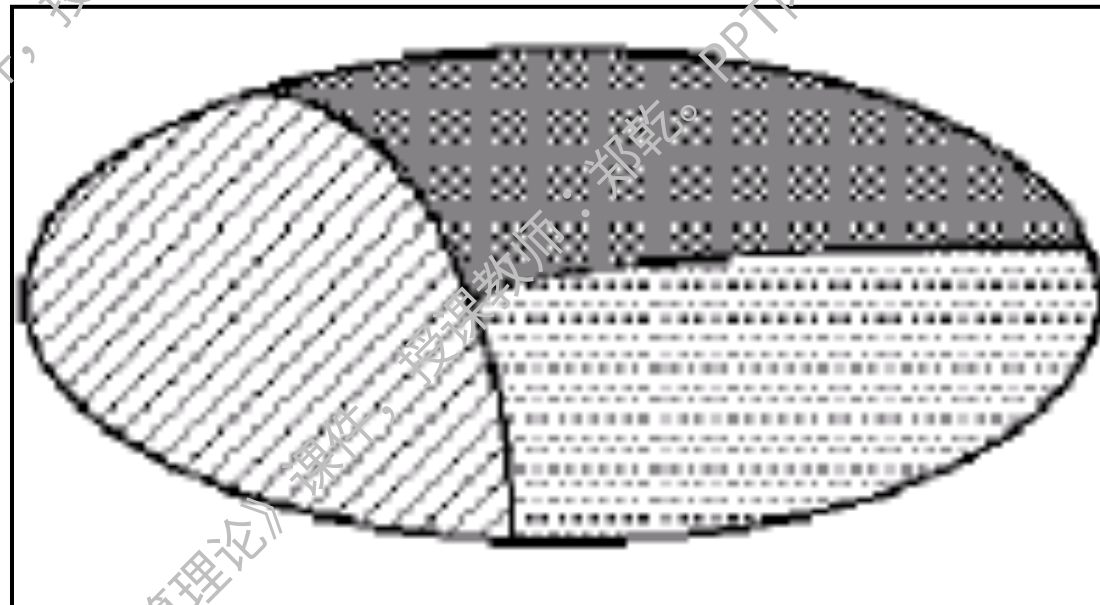
■ Partition

□ A partition of a non-empty set A is a subset Π of 2^A such that

- $\Pi \neq \emptyset$
- $\forall S, T \in \Pi$, and $S \neq T$, $S \cap T = \emptyset$
- $\cup \Pi = A$

□ Example

- The set $\{1, 2, 3\}$ has five partitions
 - $\{\{1\}, \{2\}, \{3\}\}$
 - $\{\{1, 2\}, \{3\}\}$
 - $\{\{1, 3\}, \{2\}\}$
 - $\{\{2, 3\}, \{1\}\}$
 - $\{\{1, 2, 3\}\}$



1.2 Relations and Functions



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY
ZHEJIANG UNIVERSITY

■ Ordered Pair

□ A pair (a, b) is ordered pair

$$(a, b) = (c, d) \Leftrightarrow (a = c) \wedge (b = d)$$

■ Binary Relation

□ Cartesian Product: $A \times B$

- $A \times B = \{(a, b) | a \in A \wedge b \in B\}$

□ Binary Relation

- $R \subseteq A \times B$

1.2 Relations and Functions



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY
ZHEJIANG UNIVERSITY

■ Operations of Relations

□ Inverse

$$R \subseteq A \times B \Rightarrow R^{-1} \subseteq B \times A$$

□ Composition

- Let A , B , and C be sets
- Let R be a relation from A to B and let S be a relation from B to C

$$(R \subseteq A \times B) \wedge (S \subseteq B \times C)$$

- Then R and S give rise to a relation from A to C (composition) indicated by $R \circ S$

$$R \circ S = \{(a, c) | \exists b \in B, (a, b) \in R \text{ and } (b, c) \in S\}$$

1.2 Relations and Functions



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY
ZHEJIANG UNIVERSITY

■ Example

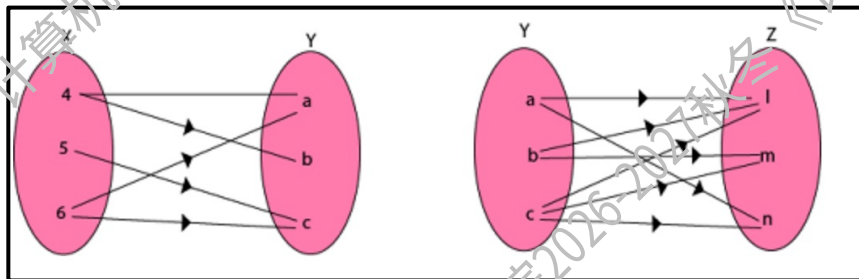
□ Let $X = \{4, 5, 6\}$, $Y = \{a, b, c\}$ and $Z = \{l, m, n\}$

□ The relation R_1 from X to Y and R_2 from Y to Z are:

$$R_1 = \{(4, a), (4, b), (5, c), (6, a), (6, c)\}$$

$$R_2 = \{(a, l), (a, n), (b, l), (b, m), (c, l), (c, m), (c, n)\}$$

➤ Find the composition of relation $R_1 \circ R_2$



➤ Find the composition of relation $R_1 \circ R_1^{-1}$

(i) The composition relation $R_1 \circ R_2$ as shown in fig:

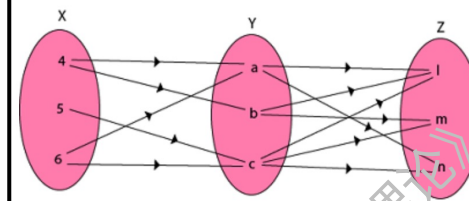


Fig : $R_1 \circ R_2$

$$R_1 \circ R_2 = \{(4, l), (4, n), (4, m), (5, l), (5, m), (5, n), (6, l), (6, m), (6, n)\}$$

1.2 Relations and Functions



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY
ZHEJIANG UNIVERSITY

■ Domain and Range

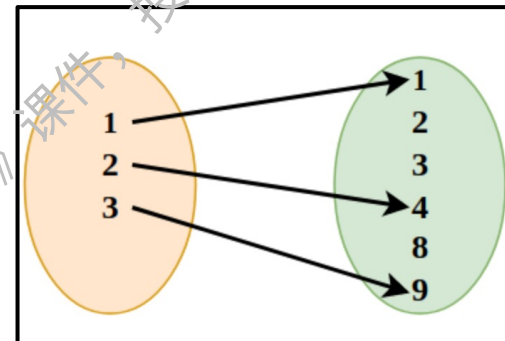
□ Domain of any relation is the set of input values of the relation

□ Range of any relation is the set of output values of the relation

□ Example

- If we take two sets A and B , and define a relation $R=\{(a,b): a \in A, b \in B\}$
- The set of values of A is called the domain of the function
- The set of values of B is called the range of the function

➤ What is the range of the relation?



1.2 Relations and Functions



■ Functions

□ **Def:** A function $f: A \rightarrow B$ must satisfy

- $f \subseteq A \times B$
- $\forall a, \exists \text{ exactly one } b \in B \text{ with } (a, b) \in f$

□ **Note**

- We write $(a, b) \in f$ as $f(a) = b$

□ **One-to-one function**

- $\forall a, b \in A \wedge a \neq b \Rightarrow f(a) \neq f(b)$

□ **Onto function**

- $\forall b \in B, \exists a \in A \text{ such that } f(a) = b$

□ **One-to-one correspondence**

- one-to-one + onto

1.2 Relations and Functions

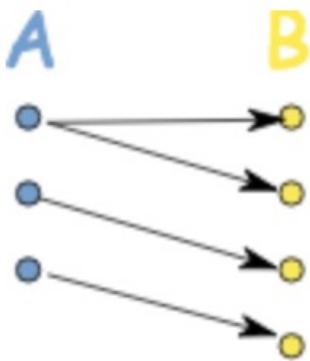


浙江大学

计算机科学与技术学院

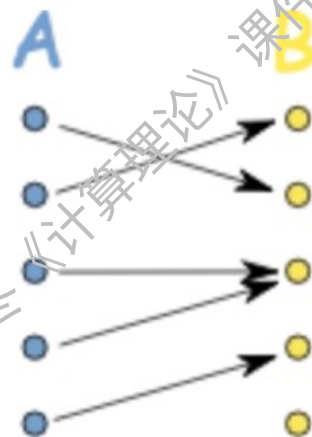
COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY
ZHEJIANG UNIVERSITY

Summary



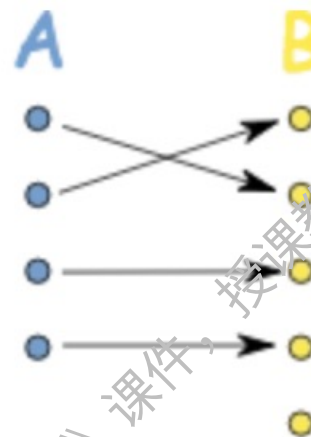
NOT a
Function

A has many B



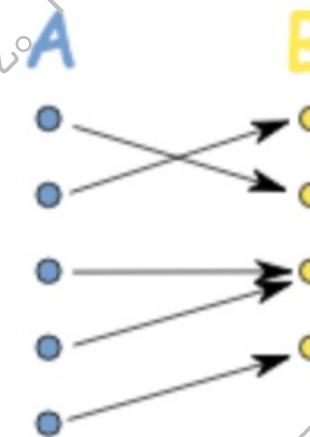
General
Function

B can have many A



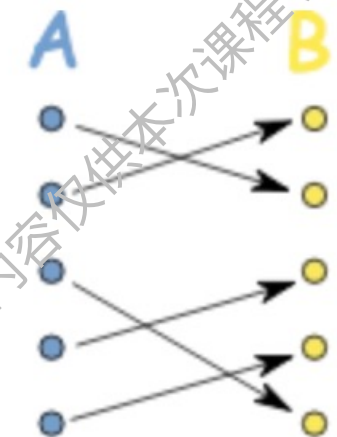
Injective
(not surjective)

B can't have many A



Surjective
(not injective)

Every B has some A



Bijjective
(injective, surjective)

A to B, perfectly

- Another type of “Not a Function”?
- Can a function be neither Injective nor Surjective?

1.3 Special Types of Binary Relations



浙江大学

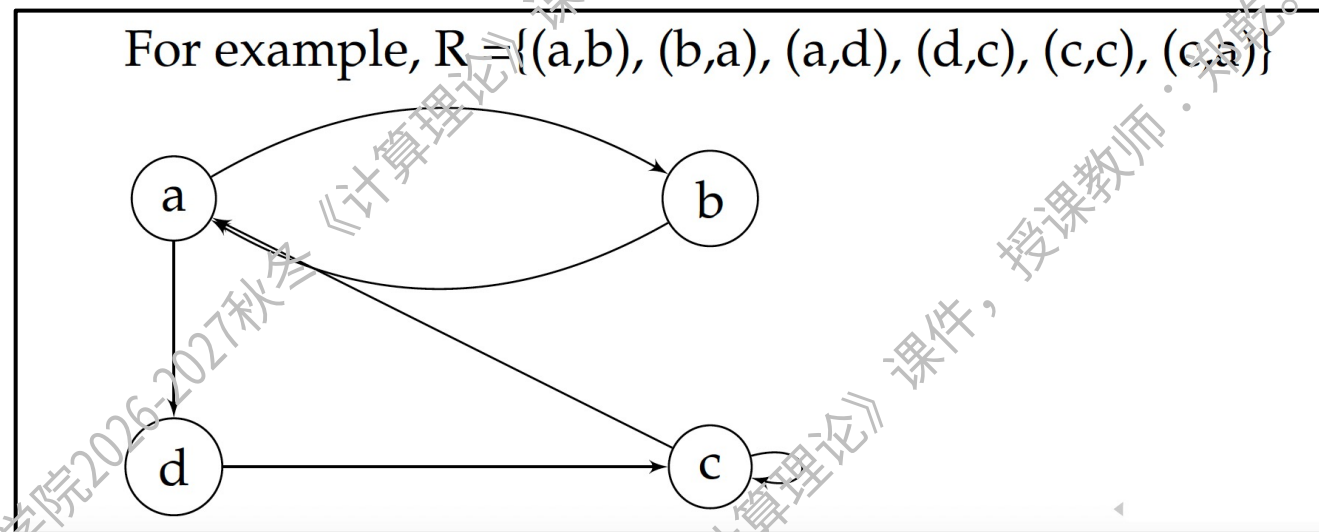
计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY
ZHEJIANG UNIVERSITY

■ Directed Graph

□ For any set A , a relation $R \subseteq A \times A$ can be represented by a **directed graph**

- A node: represented by a small circle, represent each element of A
- An arrow: is the edge of the graph, drawn from a to b iff $(a,b) \in R$
- From a node to another, there is either no edge or one edge



1.3 Special Types of Binary Relations



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY
ZHEJIANG UNIVERSITY

■ Matrix

- If R is a binary relation between sets X and Y , so $R \subseteq X \times Y$
- R can be represented by the **logical matrix** M whose row and column indices index the elements of X and Y , respectively
- The entries of M are defined by

$$m_{ij} = \begin{cases} 1 & (x_i, y_j) \in R \\ 0 & (x_i, y_j) \notin R \end{cases}$$

1.3 Special Types of Binary Relations



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY
ZHEJIANG UNIVERSITY

■ Example

- $A = \{\text{Indian, Arctic, Atlantic, Pacific}\}$, the oceans
- $B = \{\text{NA, SA, AF, EU, AS, AU, AA}\}$, the continents
- Let aRb represent that ocean a borders continent b
- Then the logical matrix for this relation R is:



Ocean borders continent							
	NA	SA	AF	EU	AS	AU	AA
Indian	0	0	1	0	1	1	1
Arctic	1	0	0	1	1	0	0
Atlantic	1	1	1	1	0	0	1
Pacific	1	1	0	0	1	1	1

1.3 Special Types of Binary Relations



■ Properties of Relations ($R \subseteq X \times Y$)

□ Reflexive: $\forall a \in A \Rightarrow (a, a) \in R$

- Consider **all** $a \in A$

□ Symmetric: $(a, b) \in R \wedge a \neq b \Rightarrow (b, a) \in R$

- Reflexive is **alternative**
- **Not necessarily** consider all pairs
- Represented by **undirected graph**

□ Antisymmetric: $(a, b) \in R \wedge a \neq b \Rightarrow (b, a) \notin R$

- Reflexive is **alternative**

□ Transitive: $(a, b) \in R, (b, c) \in R \Rightarrow (a, c) \in R$

- Reflexive is **alternative**
- **Not necessarily** consider all pairs

Let $A = \{1, 2\}$ or $A = \{1, 2, 3\}$

➤ Is $\{(1, 1), (1, 2), (2, 2)\}$ reflexive?

➤ How about $\{(2, 2)\}$?

➤ Is $\{(2, 1), (1, 2), (2, 2)\}$ symmetric?

➤ How about $\{(1, 1), (1, 2), (2, 2)\}$?

➤ Is $\{(1, 1), (1, 2), (2, 2)\}$ antisymm.?

➤ How about $\{(1, 2), (2, 1), (2, 2)\}$?

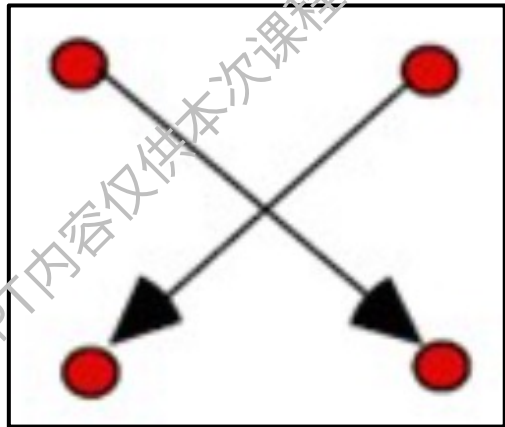
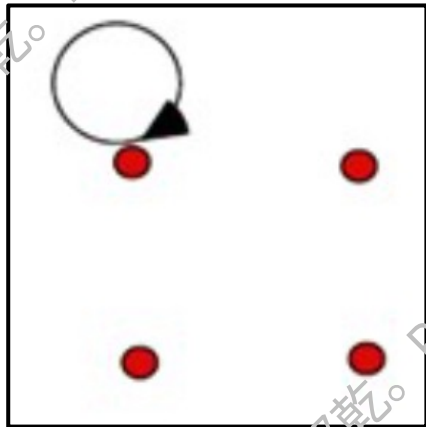
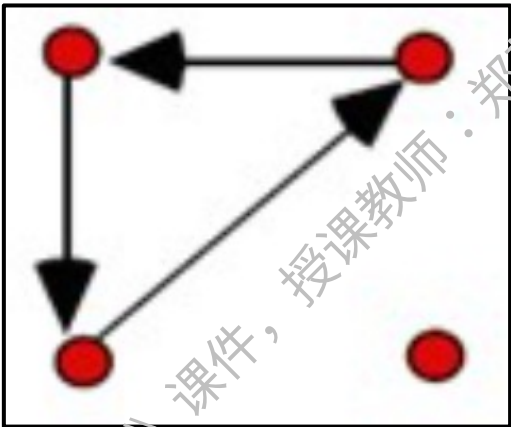
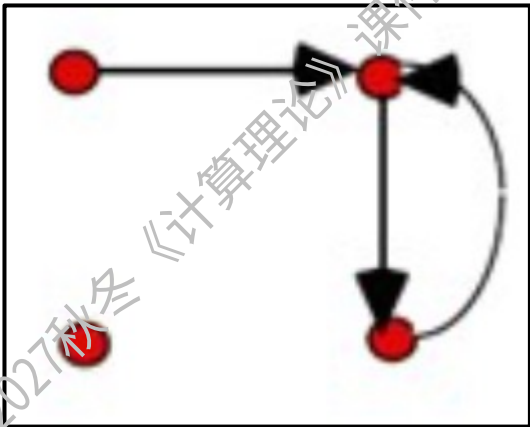
➤ Is $\{(1, 1), (1, 2), (2, 3)\}$ transitive?

➤ How about $\{(1, 2), (1, 3)\}$?

1.3 Special Types of Binary Relations



➤ Is there a relation belong **neither** or both of symmetric and antisymmetric?



	A	B	C	D
Reflexive				
Symmetric				
Antisymmetric				
transitive				

1.3 Special Types of Binary Relations



■ Equivalence Relation

□ Definition: Reflexive, Symmetric, Transitive

- A representation of an equivalent relation $R \subseteq A \times A$ as an **undirected graph** consists of several **clusters**, within clusters **each pair** is connected by a line

□ Equivalence Classes: $[a] = \{b | (a, b) \in R\}$

- The set of nodes in a cluster is an equivalence class

a, b is equivalent

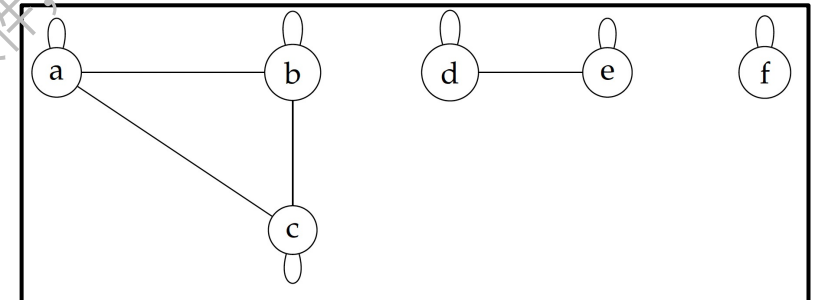
□ Example

- Let $A = \{a, b, c, d, e, f\}$, $R \subseteq A \times A$, R is represented by right graphs

- $[a] = [b] = [c] = \{a, b, c\}$

- $[d] = [e] = \{d, e\}$

- $[f] = \{f\}$



1.3 Special Types of Binary Relations



浙江大学

计算机科学与技术学院

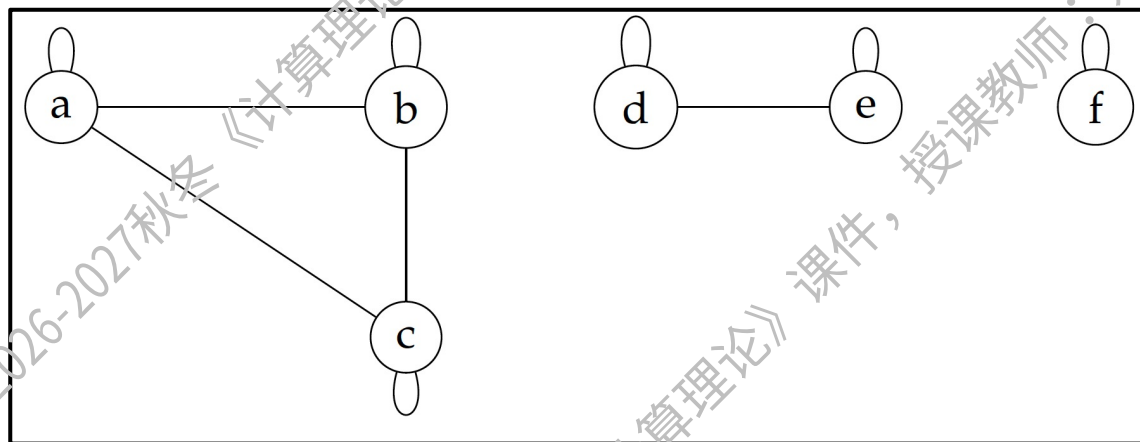
COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY
ZHEJIANG UNIVERSITY

■ Partition

□ **Theorem** Let R be an equivalence relation on a nonempty set A . Then the equivalence classes of R constitute a partition of A .

□ **Example from previous slide**

$$\{[a], [a], [f]\} = \{[b], [e], [f]\} = \{\{a, b, c\}, \{d, e\}, \{f\}\}$$



1.3 Special Types of Binary Relations



■ Partial Order

□ **Definition** Given a set A , a **partial ordering** of A is a **binary relation** $\leq$ satisfying the following axioms:

- **Reflexive:** For each $a \in A$, we have $a \leq a$
- **Transitive:** If $a \leq b$ and $b \leq c$, then $a \leq c$
- **Antisymmetric:** If $a \leq b$ and $b \leq a$, then $a = b$

□ Examples

- Let A be the set of real numbers. Define $a \leq b$ if $b - a$ is nonnegative. Then $\leq$ is partial ordering on A .
- Let S be a set, and let $P(S)$ denote the collection of all subsets of S . For $T, T' \in P(S)$, write $T \leq T'$ if $T \subseteq T'$.
This **relation makes** $P(S)$ into a partially ordered set.
- Let Z be the set of positive integers. Then Z is partially ordered by divisibility: we can define $a \leq b$ iff $a|b$

1.3 Special Types of Binary Relations



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY
ZHEJIANG UNIVERSITY

■ Partial Order

□ **Definition** Given a partially ordered set A , we say that an element $a \in A$ is a **least element** of A if $a \leq b$ for **all** $b \in A$. We say that a is a **minimal element** of A if $b \leq a$ implies $a = b$.

□ **Definition** Given a partially ordered set A , we say that an element $a \in A$ is a **greatest element** of A if $b \leq a$ for **all** $b \in A$. We say that a is a **maximal element** of A if $a \leq b$ implies $a = b$.

1.3 Special Types of Binary Relations



浙江大学

计算机科学与技术学院

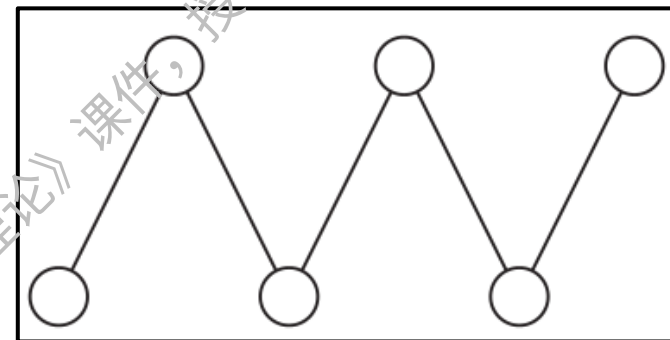
COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY
ZHEJIANG UNIVERSITY

■ Example

□ Given the collection $S = \{\{d,o\}, \{d,o,g\}, \{g,o,a,d\}, \{o,a,f\}\}$ ordered by containment

- Element $\{d,o\}$ is **minimal** as it contains no sets
- Element $\{g,o,a,d\}$ is **maximal** as there is no set containing it
- Element $\{d,o,g\}$ is **neither**, while $\{o,a,f\}$ is **both**

□ In the fence $a_1 < b_1 > a_2 < b_2 \dots$, all a_i are minimal and all b_i are maximal, as shown in the image



1.3 Special Types of Binary Relations



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY
ZHEJIANG UNIVERSITY

■ Total Order

□ **Definition** Let $(A, <)$ be a partially ordered set. We say that $\leq$ is a linear ordering or totally ordering on A if for **every pair** of elements $a, b \in A$, we have either $a \leq b$, or $b \leq a$.

- Partially ordered
 - $a \leq a$ (reflexive)
 - If $a \leq b$, and $b \leq c$, then $a \leq c$ (transitive)
 - If $a \leq b$, and $b \leq a$, then $a = b$ (antisymmetric)
- Every pair
 - For $\forall a, b$, then $a \leq b$, or $b \leq a$ (**strongly connected, called total**)

1.4 Finite and Infinite Sets



■ Equinumerous

□ **Definition:** Sets A and B equinumerous $A \sim B \Leftrightarrow \exists$ bijection $f: A \rightarrow B$

- There is no “equinumerous” function or “bijective” sets.
- There are equinumerous sets and bijective functions.
- Equinumerous sets are sets for which at least a bijective function exists.

□ **Properties:** equivalence relation

- **Reflexivity:** identity function on any A is a bijection from A to itself. $A \sim A$
- **Symmetry:** for every bijection between two sets A and B , there exists an inverse function which is bijection between B and A . $A \sim B$ implies $B \sim A$
- **Transitivity:** Given three sets A , B , and C with two bijections $f: A \rightarrow B$ and $g: B \rightarrow C$, the composition of f and g are bijection from A to C . $A \sim B$ and $B \sim C$ implies $A \sim C$

1.4 Finite and Infinite Sets



■ Cardinality

- **Definition:** the cardinality of a set is a measure of the **number** of elements of the set.
- The Cardinality of a set A is usually denoted $|A|$, $\text{card}(A)$, $\#A$
- Equinumerous sets have a one-to-one correspondence between them, and are said to have the **same cardinality**.

■ Generalized Cardinality

- Allows one to distinguish between different types of infinity

1.4 Finite and Infinite Sets



■ Finite Set

□ **Definition:** Finite sets are sets having a finite number of members

Finite sets are also known as **countable sets**.

■ Infinite Set

□ **Definition:** If a set is not finite, it is called an infinite set because the number of element in that set is **uncountable**.

■ Countable and Uncountable Infinite

□ A set is said to be countably infinite $\Leftrightarrow$ it is **equinumerous** with $\mathbb{N}$

□ S is uncountably infinite $\Leftrightarrow |S| > |\mathbb{N}|$

■ Hilbert's paradox of the Grand Hotel (countably infinite)



1.4 Finite and Infinite Sets



■ Hilbert's paradox of the Grand Hotel (countably infinite)

□ Suppose that there is a hotel that has an infinite number of rooms. As a convenience, the rooms have numbers, the first room has the number 1, the second has number 2, and so on.

- Case 1: **finite** guests cannot be taken in
- Case 2: **infinitely** new guests cannot be taken in
- Case 3: **infinitely** new buses, each contains infinitely new guests, cannot be taken in

$$n \rightarrow n + x$$

$$n \rightarrow 2n$$

$$(m, n) \rightarrow \text{prime}(m)^n$$

m represents bus number
 n represents guest number

第1章 集合、关系、语言

Ch1. Sets, Relations, and Language

1.4 Finite and Infinite Sets



■ Countable Infinite: $\aleph_0$

- A set is said to be **countably infinite** $\Leftrightarrow$ it is equinumerous with $\mathbb{N}$
- The **union** of a countable infinite collection of countable infinite sets is countably infinite
- Example: show that $\mathbb{N} \times \mathbb{N}$ is countably infinite

- Idea: Prove $\mathbb{N}$ and $\mathbb{N} \times \mathbb{N}$ are equinumerous
- **Solution 2:** find a bijection

$$f((m, n)) = \frac{1}{2} [(m + n)^2 + 3m + n]$$

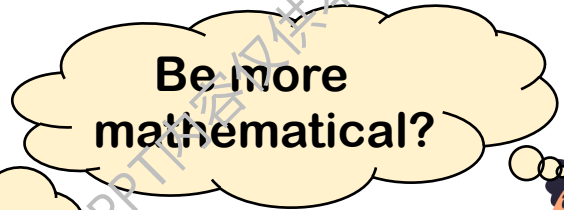
• **Solution 3:**

- Construction two injections: $f: \mathbb{N} \rightarrow \mathbb{N} \times \mathbb{N}$, $g: \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$
- Show $|\mathbb{N}| \leq |\mathbb{N} \times \mathbb{N}|$ and $|\mathbb{N} \times \mathbb{N}| \leq |\mathbb{N}|$
- $f(n) = (n, 0)$, $g((m, n)) = 2^m 3^n$

Prove $\mathbb{Q}$ is
countable (**Theorem**)

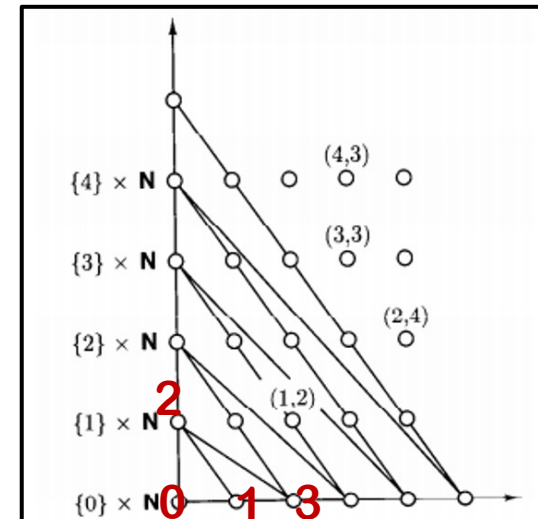


Be less
tricky?



Be more
mathematical?

Solution 1:



1.4 Finite and Infinite Sets



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY
ZHEJIANG UNIVERSITY

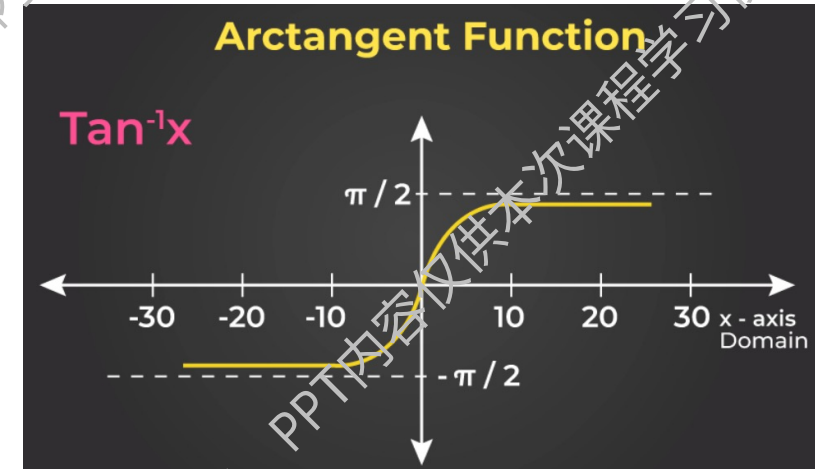
■ Theorem: $|\mathbb{R}| > |\mathbb{N}|$.

□ Proof: See next section by diagonalization

■ Question: Is $|\mathbb{R}| > |(0,1)|$?

□ Proof: Construct a bijection

$$f(x) = \frac{1}{\pi} \arctan(x) + \frac{1}{2}$$



■ Continuum hypothesis (by Cantor)

□ About the possible sizes of infinite sets.

□ "There is **no set** whose cardinality is strictly between that of the integers and the real numbers."

Let $|\mathbb{N}| = \aleph_0$, $|\mathbb{R}| = \aleph_1$,
there is no w such that $\aleph_0 < w < \aleph_1$ (**hypothesis**), and $\aleph_0 < \aleph_1$ (**theorem**)

1.4 Finite and Infinite Sets



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY
ZHEJIANG UNIVERSITY

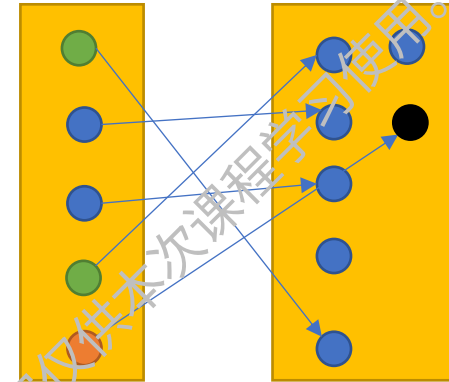
■ Cantor's Theorem

□ Let f be a map from set A to its power set $P(A)$ or 2^A . Then **card $(A) < \text{card } (P(A))$ holds** for any set A .

□ Proof: $f: A \rightarrow P(A)$ is not surjective

- Case 1: A is a finite set
- Case 2: A is an infinite set
 - We have $|A| \leq |P(A)|$. **Proof:** Consider a specific injection: $f(x) = \{x\}$, $\forall x \in A$,
 - We have $|A| \neq |P(A)|$. **Proof:** Suppose an arbitrary injection $f(x)$ to be surjective
 - Construct a set $B = \{x \in A \mid x \notin f(x)\}$, there is a t , such that $f(t) = B$
 - If $t \in B$, then by the definition of B , $t \notin f(t)$. However, $f(t) = B$.
 - If $t \notin B$, then by the definition of B , $t \in f(t)$. However, $f(t) = B$.

The power set of $\mathbb{N}$ is uncountable (**Theorem**).



Why

$B \neq \emptyset$?



1.5 Three Fundamental Proof Tech.



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY
ZHEJIANG UNIVERSITY

■ The Principle of Mathematical Induction

□ Let A be a set of natural numbers such that

- $0 \in A$, and
- For each natural number n , if $\{0, 1, 2, 3, \dots, n\} \in A$, then $n+1 \in A$

Principle of Mathematical Induction

Step 1

Prove $P(1)$
is true

Step 2

Assume
 $P(k)$ is true
for some
 $n=k$

Step 3

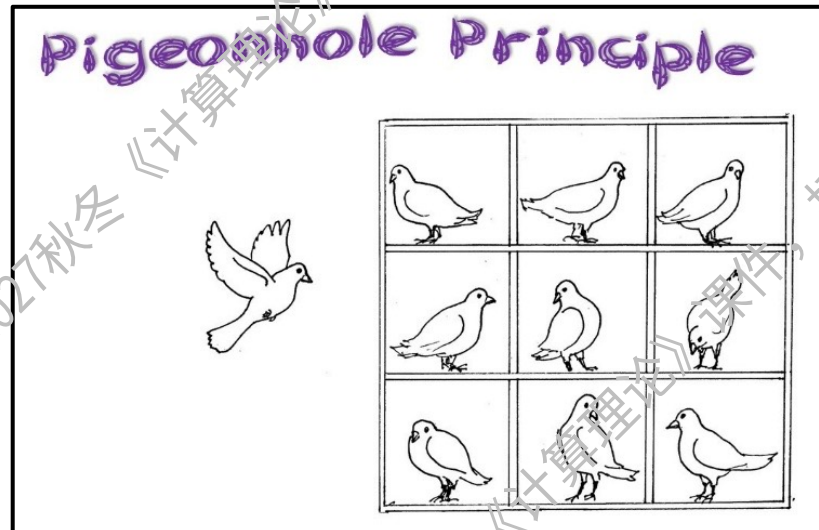
Prove $P(k+1)$
is true

1.5 Three Fundamental Proof Tech.



■ The Pigeonhole Principle

- If A and B are finite sets and $|A| > |B|$, then there is no one-to-one function (injection) from A to B
- Another Description: If m objects are placed into n bins, where $m > n$, then some bin contains at least two objects.



1.5 Three Fundamental Proof Tech.



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY
ZHEJIANG UNIVERSITY

■ Example

- Show that for any five points on a sphere, there is a closed hemisphere that contains **at least four** of them

■ Solution

- A great circle of a sphere is a circle that divides it into 2 hemispheres.
- Any 2 points on a sphere determine a great circle of it.
- Use 2 of the 5 points to find a great circle. There are 3 other points.
- By Pigeonhole Principle, at least 2 of them are contained in one of the two great circles.
- Thus, at least 4 points are contained in a closed hemisphere.

1.5 Three Fundamental Proof Tech.



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY
ZHEJIANG UNIVERSITY

■ Proof of Pigeonhole Principle

□ Basis Step.

- $|B|=0 \Rightarrow$ no function from A to $B \Rightarrow$ no (injective) function

□ Induction Hypothesis.

- Suppose $f: A \rightarrow B$, $|A| > |B|$, and $|B| \leq n$, where $n \geq 0 \Rightarrow f$ is not injective

□ Induction Step.

- Suppose $f: A \rightarrow B$, and $|A| > |B| = n+1$. Chose some $t \in A$

• Case 1: If $\exists a \in A$, such that $f(a) = f(t)$

- $\Rightarrow f$ is not injective

- Case 2: t is the only element mapped by f to $f(t)$

- $\Rightarrow |A - \{t\}| = |A| - 1 > |B| - 1 = |B - \{f(t)\}| = n$

- $\Rightarrow g: A - \{t\} \rightarrow B - \{f(t)\}$ is not injective $\Rightarrow \exists a, b \in A - \{t\}, a \neq b$, such that $g(a) = g(b)$

- $\Rightarrow f$ is not injective.

1.5 Three Fundamental Proof Tech.



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY
ZHEJIANG UNIVERSITY

■ The Diagonalization Principle

- Let R be a binary relation on a set A , and let D , the diagonal set for R , $D = \{a | a \in A \wedge (a, a) \notin R\}$. For each $a \in A$, let

$$R_a = \{b | b \in A \wedge (a, b) \in R\}$$

- Then D is distinct from each R_a

■ Example

- Let us consider the relation

$$R = \{(a, b), (a, d), (b, b), (b, c), (c, c), (d, b), (d, c), (d, e), (d, f), (e, e), (f, a), (f, d), (f, e)\}$$

- We have:

- $D = \{a, d, f\}$
- $R_a = \{b, d\}$, $R_b = \{b, c\}$, $R_c = \{c\}$, $R_d = \{b, c, e, f\}$,
- $R_e = \{e, f\}$, $R_f = \{a, c, d, e\}$



	a	b	c	d	e	f	
a		×		×			R_a
b		×	×				R_b
c			×				
d		×	×		×	×	
e					×	×	
f	×		×	×	×		R_f
	a	×	×	d	×	f	D

1.5 Three Fundamental Proof Tech.



■ Cantor's Theorem

□ Let f be a map from set A to its power set $P(A)$ or 2^A .

Then $\text{card}(A) < \text{card}(P(A))$ holds for any set A

■ Proof:

□ ...

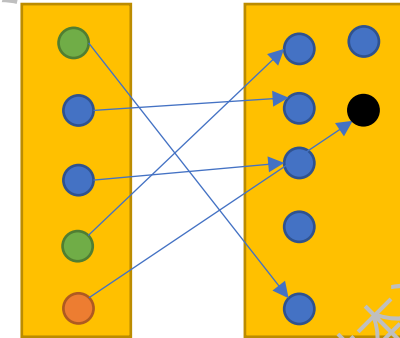
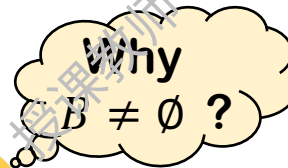
□ We have $|A| \neq |P(A)|$.

□ **Proof:** Suppose a given injection $f(x)$ to be surjective

Construct a set $B = \{x \in A \mid x \notin f(x)\}$, there is a t , such that

$$f(t) = B$$

- If $t \in B$, by the definition of B , $t \notin f(t)$, contradict
- If $t \notin B$, by the definition of B , $t \in f(t)$, contradict



	a	b	c	d	e	f
f(a)		×		×		
f(b)		×	×			
f(c)			×			
f(d)		×	×		×	×
f(e)					×	×
f(f)	×		×	×	×	
B	a	×	×	d	×	f

1.5 Three Fundamental Proof Tech.



■ Theorem: $|\mathbb{R}| > |\mathbb{N}|$.

□ **Proof Goal:** Show that there is no bijection $f: \mathbb{N} \rightarrow \mathbb{R}$

- Assume there is such a bijection
- Derive a contradiction by showing that f cannot be surjective

□ **Proof Idea:**

- List an infinite sequence of real numbers, $r_0, r_1, r_2, \dots$ by setting $r_i = f(i)$
- We show that we can always find a real number d such that

$$\forall n \in \mathbb{N}: r_n \neq d \Leftrightarrow r_0 \neq d, r_1 \neq d, r_2 \neq d, r_3 \neq d, \dots$$

- Build the real number d out of infinitely many “pieces”, with one piece for each r_i
 - Choose the 0th piece such that $r_0 \neq d$
 - Choose the 1st piece such that $r_1 \neq d$
 - Choose the 2nd piece such that $r_2 \neq d$
 - ...

1.5 Three Fundamental Proof Tech.



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY
ZHEJIANG UNIVERSITY

■ **Theorem:** $|\mathbb{R}| > |\mathbb{N}|$.

□ **Proof Idea:**

- Define d via its decimal representation: choose the digits of d as follows:
 - The 0th digit of d is not the same as 0th digit of r_0
 - The 1st digit of d is not the same as 1st digit of r_1
 - The 2nd digit of d is not the same as 2nd digit of r_2
 - ...
- So $d \neq r_i$ for any $i \in \mathbb{N}$

1.5 Three Fundamental Proof Tech.



■ **Theorem:** $|\mathbb{R}| > |\mathbb{N}|$.

Proof:

□ If r_i is a real number, define $r_i[n]$ as follows

- $r_i[0]$ is the integer part of r_i
- $r_i[n]$ is the n -th decimal digit of r_i , if $n > 0$

• **Examples:**

- $\pi[0] = 3, \pi[1] = 1, \pi[2] = 4, \pi[3] = 1$
- $(-e)[0] = -2, (-e)[1] = -7, (-e)[2] = 1, (-e)[3] = 8$
- $5[0] = 5, 5[1] = 0, 5[2] = 0, 5[3] = 0$

	[0]	[1]	[2]	[3]	[4]	[5]	...
r_0	8.	6	7	5	0	0	...
r_1	3.	1	4	1	5	9	...
r_2	0.	1	2	3	5	8	...
r_3	-1.	0	0	0	0	0	...
r_4	2.	7	1	8	2	8	...
r_5	1.	6	1	8	0	3	...
...	...	...	...	...	...	...	...

1.5 Three Fundamental Proof Tech.



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY
ZHEJIANG UNIVERSITY

■ **Theorem:** $|\mathbb{R}| > |\mathbb{N}|$.

Proof:

□ Define $d[n] = \begin{cases} 1 & \text{if } r_n[n] = 0 \\ 0 & \text{otherwise} \end{cases}$

□ Now, $d \neq r_i$ for any $i \in \mathbb{N}$

- If $r_i[i] = 0$, then $d[i] = 1$, so $d \neq r_i$
- If $r_i[i] \neq 0$, then $d[i] = 0$, so $d \neq r_i$

□ f cannot be surjective

	[0]	[1]	[2]	[3]	[4]	[5]	...
r_0	8.	6	7	5	0	0	...
r_1	3.	1	4	1	5	9	...
r_2	0.	1	2	3	5	8	...
r_3	-1.	0	0	0	0	0	...
r_4	2.	7	1	8	2	8	...
r_5	1.	6	1	8	0	3	...
...	...	...	...	...	...	...	...

d	0.	0	0	1	0	0	...
-----	----	---	---	---	---	---	-----

1.5 Three Fundamental Proof Tech.



■ Theorem: (G. Cantor 1845-1918) The set $2^{\mathbb{N}}$ is uncountable

□ **Proof Idea:** Assume for contradiction that $f: A \rightarrow B$ is surjective. We

will find $d \in B$ such that $f(a) \neq d$ for any $a \in A$

- Construct d based on “pieces”, one piece taken from each $a \in A$
- Construct d such that a -th “piece” of d disagrees with a -th “piece” of $f(a)$
- Conclude that $f(a) \neq d$ for any $a \in A$

□ **Proof**

- Suppose that $2^{\mathbb{N}}$ is countably infinite
- Enumerate all members of $2^{\mathbb{N}}$ as $2^{\mathbb{N}} = \{R_0, R_1, R_2, \dots\}$
- Construct set $D = \{n \in \mathbb{N} | n \notin R_n\}$
- Since $D \subseteq 2^{\mathbb{N}}$, it should appear in $\{R_0, R_1, R_2, \dots\}$, but D cannot equal to R_i

1.5 Three Fundamental Proof Tech.



■ Example

- Considering the set T of all **infinite** sequences of binary digits (i.e. each digit is zero or one). Is T countable?



What if the sequences is finite?

■ Proof

- The set $\{1, 4\}$ corresponds to an infinite sequence of 0s and 1s that has 1s at position 1 and 4, and 0s everywhere else, i.e., the sequence 0, 1, 0, 0, 1, 0, 0, 0, 0

$f(0) = \{1, 4\}$
 $f(1) = \{0, 2, 3, 4\}$
 $f(2) = \emptyset$
 $f(3) = \{5, 6, 7\}$
 $f(4) = \mathbb{N}$
 $f(5) = \mathbb{E}$
 $f(6) = \{1, 3, 5, 7\}$
 $f(7) = \{0, 2, 4\}$

	0	1	2	3	4	5	6	7	...
0	0	1	0	0	1	0	0	0	...
1	1	1	1	1	1	0	0	0	...
2	0	0	1	0	0	0	0	0	...
3	0	0	0	1	0	1	1	1	...
4	1	1	1	1	1	1	1	1	...
5	1	0	1	0	1	1	1	0	...
6	0	1	0	1	0	1	1	1	...
7	1	0	1	0	1	0	0	1	...
...	...	...	...	...	...	...	...	...	...

1	2
2	4
3	8
4	16
5	32
6	64

1.6 Closure



■ Intuitive Idea

- Natural numbers $\mathbb{N}$ are closed under $+$, i.e., for given two natural numbers n, m , we always have $n + m \in \mathbb{N}$
- Natural numbers $\mathbb{N}$ are **not** closed under subtraction $-$, i.e., there are two natural numbers n, m , such that $n - m \notin \mathbb{N}$, for example, $1, 2 \in \mathbb{N}$ but $1 - 2 = -1 \notin \mathbb{N}$
- Integers $\mathbb{Z}$ are closed under $-$.
- $\mathbb{Z}$ is the **smallest** set containing $\mathbb{N}$ and closed under subtraction $-$

■ Example (two sets and their operation)

- The set $\mathbb{Z}$ is called **a closure of $\mathbb{N}$ under subtraction $-$**

1.6 Closure



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY
ZHEJIANG UNIVERSITY

■ Closures of Relations

- Given any binary relation R , one can form closures with respect to any combinations of the properties: **Reflexive, Symmetric, Transitive**

■ Note

- Reflexive, transitive closure of R is usually denoted as R^*

■ Definition

- Let $R \subseteq A \times A$ be a directed graph. The reflexive transitive closure of R is the relation

$$R^* = \{(a, b) | a, b \in A \text{ and there is path from } a \text{ to } b \text{ in } R\}$$

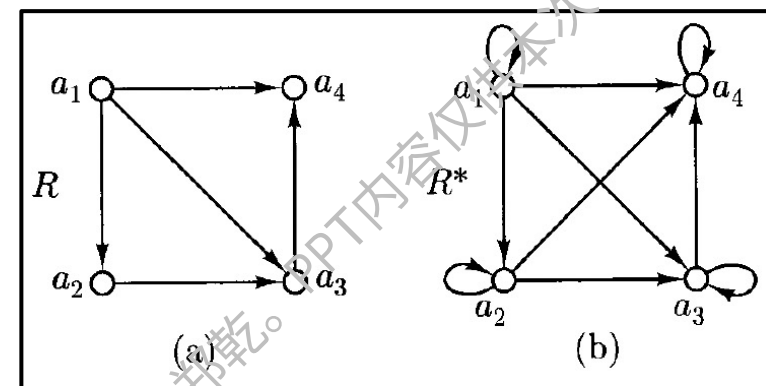
1.6 Closure



■ Example (two sets and their operation)

□ Consider two directed graphs as shown left. Observe that:

- $R^* = R \cup \{(a_i, a_j) \mid i = 1, 2, 3, 4\} \cup \{a_2, a_4\}$
- $R \subseteq R^*$
- R^* is **reflexive** and **transitive**
- R is neither
- R^* is **smallest set** containing R that is **reflexive** and **transitive**



□ We call such relation R^* the **reflexive, transitive closure** of R

1.6 Closure



■ Example: The **Transitive** Closure

- The **smallest** relation that includes R and its **transitive** (usually called R^+)
- More formally: R^+ is a relation such that
 - $R \subseteq R^+$
 - R^+ is **transitive**
 - $\forall R', R \subseteq R'$ and R' is transitive $\Rightarrow R^+ \subseteq R'$

- Can you define the relation of “parent-of” ?
- What is symmetric, transitive closure of “parent-of” is?

-
- The diagram illustrates the process of translating a problem into computational models. It shows a flow from 'Problem' to 'Modeling' to 'Coding' to 'Sets of Symbols' to 'Languages' to 'Computational Models'. The steps are connected by arrows indicating the direction of translation.
- ```

graph LR
 Problem[Problem] --> Modeling[Modeling]
 Modeling --> Coding[Coding]
 Coding --> Symbols[Sets of Symbols]
 Symbols --> Languages[Languages]
 Languages <--> Models[Computational Models]

```
- Problem**
- Modeling**
- Coding**
- Sets of Symbols**
- Languages**
- Computational Models**
- Compile & Assemble**
- Example:**
- |   | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | ... |
|---|---|---|---|---|---|---|---|---|-----|
| 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | ... |
| 1 | 1 | 0 | 1 | 1 | 1 | 0 | 0 | 0 | ... |
| 2 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | ... |
| 3 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | ... |
| 4 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | ... |
| 5 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | ... |
| 6 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | ... |
| 7 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | ... |
| : | : | : | : | : | : | : | : | : | :   |
- Formal Definitions:**
- $f(0) = \{1, 4\}$
  - $f(1) = \{0, 2, 3, 4\}$
  - $f(2) = \emptyset$
  - $f(3) = \{5, 6, 7\}$
  - $f(4) = \mathbb{N}$
  - $f(5) = \mathbb{E}$
  - $f(6) = \{1, 3, 5, 7\}$
  - $f(7) = \{0, 2, 4\}$

# 1.7 Alphabet and Language



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY  
ZHEJIANG UNIVERSITY

## ■ Alphabet

□ **Definition:** Any **finite** set is called an alphabet. Elements of the alphabet are called **symbols** of the alphabet.

□ **Notation:** We use a symbol  $\Sigma$  to denote the alphabet

□ **Examples:**

- $\Sigma = \{\leftarrow, \emptyset, \nexists, \cong, \sqrt[4]{\phantom{x}}\}$
- $\Sigma = \{a, b, c\}$
- $\Sigma = \{n \in \mathbb{N} : n \leq 10^5\}$
- $\Sigma = \{0, 1\}$  is called a binary alphabet

➤ Is  $\emptyset$  an alphabet?

# 1.7 Alphabet and Language



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY  
ZHEJIANG UNIVERSITY

## ■ Strings

- We call **finite** sequences of the alphabet  $\Sigma$  **words** or **strings over  $\Sigma$**
- We denote by  $\epsilon$  the empty string **over  $\Sigma$**
- We denote by  $\Sigma^*$  the set of all strings **over  $\Sigma$**
- Elements of  $\Sigma^*$  are called strings **over  $\Sigma$**
- We write  $w \in \Sigma^*$  to express that  $w$  is a string **in  $\Sigma^*$**
- String length
- Examples:

- Let  $\Sigma = \{a, b\}$ , we will write

➤ Is  $\emptyset = \Sigma^*$  for some  $\Sigma$  ?

tation as

$aaa, bbb, ab, abab$

# 1.7 Alphabet and Language



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY  
ZHEJIANG UNIVERSITY

## ■ Operation of Strings:

### □ Concatenation: $x \circ y$ or $xy$

- Substring, suffix, prefix
- Example:  $\forall w, we = ew = w$

### □ String exponentiation

- $w^0 = e$ , the empty string
- $w^{i+1} = w^i \circ w$ , for each  $i \geq 0$

### □ Reversal

- If  $w$  is a string of **length** 0, then  $w^R = w = e$
- If  $w$  is a string of **length**  $n + 1 > 0$ , then  $w = ua$  for some  $a \in \Sigma$ , and  $w^R = au^R$

Definition by induction

# 1.7 Alphabet and Language



## ■ Language: Set of Strings

□ Alphabet:  $\Sigma$

□ The set of all strings:  $\Sigma^* (e \in \Sigma)$

□ Language:  $L \subseteq \Sigma^*$

□  $\emptyset, \Sigma$ , and  $\Sigma^*$  are languages

□ Finite Language: by listing all the strings

□ Infinite Language: specify by the following scheme

$$L = \{w \in \Sigma^* : w \text{ has property } P\}$$

■ Example:  $L = \{ab, aabb, aaabbbb, \dots\} = \{a^n b^n | n \geq 1\}$



**Theorem:** If  $\Sigma$  is a finite alphabet, then  $\Sigma^*$  is countably infinite set.

➤ If  $\Sigma$  is a finite alphabet, then  $\Sigma^*$  is countably infinite or uncountable?

**Example**

**Considering the set  $T$  of all infinite sequences of binary digits (i.e. each digit is zero or one). Is  $T$  countable?**

## Proof

- The set  $\{1, 4\}$  corresponds to an infinite sequence of 0s and 1s that has 1s at position 1 and 4, and 0s everywhere else, i.e., the sequence 0, 1, 0, 0, 1, 0, 0, 0, 0

$$f(0) = \{1, 4\}$$

$$f(1) = \{0, 2, 3, 4\}$$

$$f(2) = \emptyset$$

$$f(3) = \{5, 6, 7\}$$

$$f(4) = \mathbb{N}$$

$$f(5) = \mathbb{E}$$

$$f(6) = \{1, 3, 5, 7\}$$

$$f(7) = \{0, 2, 4\}$$

[illegible]

# 1.7 Alphabet and Language



**Theorem:** If  $\Sigma$  is a finite alphabet, then  $\Sigma^*$  is countably infinite set.

■ **Proof:** Construct a bijection  $f: \mathbb{N} \rightarrow \Sigma^*$

□ Fix some ordering of the alphabet, say  $\Sigma = \{a_1, a_2, \dots, a_n\}$ .

□ The member of  $\Sigma^*$  can be enumerated in the following way

- For each  $k \geq 0$ , all string of length  $k$  are enumerated before all strings of length  $k + 1$
- The  $n^k$  strings of length exactly  $k$  are enumerated **lexicographically**

□ For example, if  $\Sigma = \{0,1\}$ , the order would be as follows:

$\epsilon, 0, 1, 00, 01, 10, 11, 000, 001, 010, 011, \dots$

# 1.7 Alphabet and Language



## ■ How many strings over a non-empty alphabet?

- Countably infinite
- All finite sequences of finite symbols (**alphabet**) is countably infinite.

## ■ Is a language countable over a non-empty alphabet?

- Any language over any alphabet is countable.
- Language is a subset of all finite sequences of finite symbols (**alphabet**).

## ■ How many languages over a non-empty alphabet?

- $|\mathbb{R}|$
- The cardinality of the power set (**language number**) of a countably infinite set (**string**) is  $|\mathbb{R}|$ .

# 第1章 集合、关系、语言

## Ch1. Sets, Relations, and Language

# 1.7 Alphabet and Language



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY  
ZHEJIANG UNIVERSITY

## ■ Operations of Languages:

□ Union, Intersection, Difference, Complement ( $\bar{L} = \Sigma^* - L$ )

□ Exponentiation

- $L^0 = \{e\}$
- $L^{i+1} = LL^i$ , for each  $i \geq 0$

□ Concatenation

$$L_1L_2 = \{w_1w_2 | w_1 \in L_1 \wedge w_2 \in L_2\}$$

□ Example

- $L_1 = \{w \in \{0,1\}^* : w \text{ has an even number of } 0\text{'s}\}$
- $L_2 = \{w \in \{0,1\}^* : w \text{ starts with a } 0, \text{ the rest symbol are of } 1\text{'s}\}$
- $L_1L_2 = \{w \in \{0,1\}^* : w \text{ has an odd number of } 0\text{'s}\}$

➤ If  $L = \{a, b\}$ , can  $ab \in LL$ ?

# 1.7 Alphabet and Language



## ■ Operations of Languages:

### □ Kleene Star

$$L^* = \{w \in \Sigma^*: w = w_1 \cdots w_k, k \geq 0, w_1, \dots, w_k \in L\} = L^0 \cup L^1 \cup L^2 \cup \dots$$

### □ We write $L^+$ for the language $LL^*$ . Equivalently, $L^+$ is the language

$$L^+ = \{w \in \Sigma^*: w = w_1 \cdots w_k, k \geq 1, w_1, \dots, w_k \in L\} = L^1 \cup L^2 \cup L^3 \cup \dots$$

### □ Note that $L^+$ can be considered as the closure of $L$ under the function of **concatenation**. That is, $L^+$ is the **smallest** language that include $L$ and all strings that are concatenations of strings in $L$

- Can  $e \in L^+$  for some  $\Sigma$ ?
- When we have  $L^+ = L^*$
- Can  $L^*$  or  $\emptyset^*$  equal to  $\emptyset$ ?

# 1.7 Alphabet and Language



## ■ Remark

□ The use of  $\Sigma^*$  to denote the set of all strings over  $\Sigma$  is consistent with the notation for the Kleene star of  $\Sigma$ .

□  $\emptyset^* = \{e\}$

□  $L^+ = LL^*$

□ For any language  $L$ ,  $(L^*)^* = L^*$ ;  $L\emptyset = \emptyset L = \emptyset$

## ■ Example

□  $L = \{w \in \{0,1\}^* : w \text{ has an unequal number of 0's and 1's}\}$ . Then  $L^* = \{0,1\}^*$

• Hint:  $L_1 \subseteq L_2 \Rightarrow L_1^* \subseteq L_2^* \quad \{0,1\} \subseteq L$

•  $L^* = (L^*)^*$

# 1.8 Finite Representations of Languages



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY  
ZHEJIANG UNIVERSITY

- We can **represent** a finite language by **finite means**
  - for example listing all its elements
- Languages are often **infinite**
  - and so a natural question arises: if a finite representation is possible and when it is possible when a **language is infinite**
- The representation of languages by **finite** specifications is a central issue of the **theory of computation**
- Of course we have to define first formally what do we mean by representation by **finite specifications**, or more precisely by a **finite representation**

# 1.8 Finite Representations of Languages



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY  
ZHEJIANG UNIVERSITY

## ■ Finite Representations

- must be a string
- different languages to have different representations

■ **Example: Let  $L = \{w \in \{0,1\}^* : w \text{ has two or three occurrences of 1, the first and second of which are not consecutive}\}$ .**

- The language can be described using only singleton sets and the symbols  $\cup$ ,  $\circ$ , and  $*$  as

$$\{0\}^* \circ \{1\} \circ \{0\}^* \circ \{0\} \circ \{1\} \circ \{0\}^* \left( (\{1\} \circ \{0\}^*) \cup \emptyset^* \right)$$

- The language can be written simply as

$$0^* 1 0^* 0 1 0^* (1 0^* \cup \emptyset^*)$$

**Regular  
Expression**

# 1.8 Finite Representations of Languages



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY  
ZHEJIANG UNIVERSITY

■ **Definition:** The **regular expressions** are all strings over the alphabet  $\Sigma \cup \{ (, ), \cup, * \}$  that can be obtained as follows.

1.  $\emptyset$  and  $\{x\} (\forall a \in \Sigma)$  is a regular expression.
2. If  $\alpha$  and  $\beta$  are regular expressions, then so are  $(\alpha\beta)$ ,  $(\alpha \cup \beta)$ ,  $\alpha^*$
3. Nothing is regular expression unless it follows from 1. through 2.

# 1.8 Finite Representations of Languages



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY  
ZHEJIANG UNIVERSITY

■ **Definition:** We define a  $\mathcal{R}$  of **regular expressions** over an alphabet  $\Sigma$  as follows:  $\mathcal{R} \subseteq (\Sigma \cup \{ (, ), \emptyset, \cup, * \})^*$  and  $\mathcal{R}$  is the smallest set such that

□  $\emptyset \in \mathcal{R}$  and  $\Sigma \subseteq \mathcal{R}$

□ If  $\alpha, \beta \in \mathcal{R}$ , then

•  $\{\alpha\beta\} \in \mathcal{R}, (\alpha \cup \beta) \in \mathcal{R}, \alpha^* \in \mathcal{R}$

■ **Examples:**

□  $L = \{ab, aabb, aaabbb, \dots\} = \{a^n b^n | n \geq 1\}$

□  $a^* b^*, a^* \cup b^*, a(a^* \cup b^*), (a^* \cup b^*)a(a^* \cup b^*), aaaaaa a^*$

# 1.8 Finite Representations of Languages



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY  
ZHEJIANG UNIVERSITY

- We use the regular expressions from the set  $\mathcal{R}$  as a representation of languages
- Languages represented by **regular expressions** are called **regular languages**.

➤ Can we finitely represent all languages over an alphabet?

Representations  
( $\Sigma^*$ , countable)

Languages  
( $2\Sigma^*$ , uncountable)

⇒  $\exists$  undescribable languages!

# 1.8 Finite Representations of Languages



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY  
ZHEJIANG UNIVERSITY

## ■ Regular expressions & languages

□ **Definition:** The representation relation between regular expressions and languages they represent is established by a function  $\mathcal{L}$  such that :

- If  $a \in \mathcal{R}$  is any regular expression, then  $\mathcal{L}(a)$  is the language represented by  $a$

### □ Formal definition

- The function  $\mathcal{L}: \mathcal{R} \rightarrow 2^{\Sigma^*}$ , is defined **recursively** as follows

- $L(\emptyset) = \emptyset, L(a) = \{a\}$  for all  $a \in \Sigma$
- If  $\alpha, \beta \in \mathcal{R}$ , then
  - $\mathcal{L}(\alpha\beta) = \mathcal{L}(\alpha) \circ \mathcal{L}(\beta)$  concatenation
  - $\mathcal{L}(\alpha\beta) = \mathcal{L}(\alpha) \cup \mathcal{L}(\beta)$  union
  - $\mathcal{L}(a^*) = \mathcal{L}(a)^*$  Kleene's Star

# 1.8 Finite Representations of Languages



■ **Example:** what is  $\mathcal{L}\left(\left((a \cup b)^* a\right)\right)$  ?

□  $\mathcal{L}\left(\left((a \cup b)^* a\right)\right) = \mathcal{L}\left((a \cup b)^*\right) \mathcal{L}(a) = \mathcal{L}(a \cup b)^* \mathcal{L}(a) = \left(\left(\mathcal{L}(a) \cup \mathcal{L}(b)\right)^* \mathcal{L}(a)\right)$

□  $= (\{a, b\})^* \{a\} = \{w \in \{a, b\}^*: w \text{ ends with an } a\}$

■ **Regular Expression Identities**

- $SR \neq RS$
- $S \cup R = R \cup S$
- $R(ST) = (RS)T$
- $R(S \cup T) = RS \cup RT, (R \cup S)T = RT \cup ST$
- $\emptyset^* = \{e\}$
- $(R^*)^* = R^*$
- $(R^* S^*)^* = (R \cup S)^*$
- $(\{e\} \cup R)^* = R^*$

# 1.8 Finite Representations of Languages



■ **Example:** what language is represented by  $c^*(a \cup (bc^*)^*)^*$

□ **Solution:** based on  $a$

- Suppose there is no  $a$ , we have  $L = c^*(bc^*)^* = \{b, c\}^*$
- Suppose there is no  $a$  and no  $c^*$ , we have  $L = e \cup b\{b, c\}^*$
- Suppose there is at least one  $a$ , we find all  $a$ , and rewrite the string as

$x_1 a x_2 a x_3 \dots a x_k$

- Considering  $x_1$ ,  $x_1 = c^*(bc^*)^* = \{b, c\}^*$ 
  - Considering  $x_i$ ,  $x_i = (bc^*)^* = e \cup b\{b, c\}^*$
- $L = \{w \in \{a, b, c\}^* : \text{not have the substring } ac\}$

# 1.8 Finite Representations of Languages



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY  
ZHEJIANG UNIVERSITY

## ■ Remark:

- Every language that can be represented by a regular expression can be represented by **infinitely** many of them.
- The **class** of regular languages over an alphabet  $\Sigma$  is defined to consist of **all** languages  $L$  such that  $L = L(a)$  for some regular expression  $a$  over  $\Sigma$ , i.e., the class of regular languages over an alphabet  $\Sigma$  is precisely the **Union, Concatenation, and Kleene Star closure** of the set of languages  $\{\{\sigma\} : \sigma \in \Sigma\} \cup \{\emptyset\}$
- The regular expressions are an inadequate specification method in general. For example,  $\{0^n 1^n : n \geq 0\}$  cannot be described by regular expressions.

# 1.8 Finite Representations of Languages



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY  
ZHEJIANG UNIVERSITY

## ■ Two important and useful means of representing languages:

- **language recognition device**: to answer questions of the form “Is string  $w$  a member of  $L$ ?”
- **language generators**: regular expression

## ■ Example

- **language recognition device**:  $\{w \in \{0,1\}^* : w \text{ does not have } 111 \text{ as a substring}\}$ .
  - by reading strings, a symbol at a time, from left to right, might operate like this:
    - Keep a count, which starts at zero and is set back to zero every time a 0 is encountered in the input;
    - add one every time a 1 is encountered in the input;
    - stop with a **No** answer if the count ever reaches three, and stop with a **Yes** answer if the whole string is read without the count reaching three.

# 1.8 Finite Representations of Languages



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY  
ZHEJIANG UNIVERSITY

## ■ Example

### □ language generators

- An alternative and somewhat orthogonal method for specifying a language is to describe how a generic specimen in the language is produced.
- For example, a regular expression such as  $(e \cup b \cup bb)(a \cup ab \cup abb)^*$  may be viewed as a way of generating members of a language:
  - To produce a member of  $L$ , first write down either nothing, or  $b$ , or  $bb$ ;
  - then write down  $a$  or  $ab$ , or  $abb$ , and do this any number of times, including zero;
  - all and only members of  $L$  can be produced in this way.

# Exercises



浙江大学

计算机科学与技术学院

COLLEGE OF COMPUTER SCIENCE AND TECHNOLOGY  
ZHEJIANG UNIVERSITY

1.7.4. Show each of the following.

- (a)  $\{e\}^* = \{e\}$
- (b) For any alphabet  $\Sigma$  and any  $L \subseteq \Sigma^*$ ,  $(L^*)^* = L^*$ .
- (c) If  $a$  and  $b$  are distinct symbols, then  $\{a, b\}^* = \{a\}^* (\{b\} \{a\}^*)^*$ .
- (d) If  $\Sigma$  is any alphabet,  $e \in L_1 \subseteq \Sigma^*$  and  $e \in L_2 \subseteq \Sigma^*$ , then  $(L_1 \Sigma^* L_2)^* = \Sigma^*$ .
- (e) For any language  $L$ ,  $\emptyset L = L \emptyset = \emptyset$ .

1.7.6. Under what circumstances is  $L^+ = L^* - \{e\}$ ?

$L^+ = L^* - \{e\}$  exactly when  $e \notin L$ .

1.8.3. Let  $\Sigma = \{a, b\}$ . Write regular expressions for the following sets:

- (a) All strings in  $\Sigma^*$  with no more than three  $a$ 's.
- (b) All strings in  $\Sigma^*$  with a number of  $a$ 's divisible by three.
- (c) All strings in  $\Sigma^*$  with exactly one occurrence of the substring  $aaa$ .

(a)  $b^* \cup b^* a b^* \cup b^* a b^* a b^* \cup b^* a b^* a b^* a b^*$

(b)  $b^* (a b^* a b^* a b^*)^*$

(c)  $(a \cup a a \cup b^*)^* a a a (b b^* (a \cup a a \cup b^*))^*$

1.8.5. Which of the following are true? Explain.

- (a)  $baa \in a^* b^* a^* b^*$
- (b)  $b^* a^* \cap a^* b^* = a^* \cup b^*$
- (c)  $a^* b^* \cap b^* c^* = \emptyset$
- (d)  $abcd \in (a(cd)^* b)^*$

True

True

False

False